



Logging Guide

Introduction

VisUAL supports logging the emulator state both via the command line and via the GUI. Logs are saved as XML files to allow easy parsing by external tools. The XML structure is explained below.

While the GUI and the command line modes support logging equally, the command line interface is more powerful in that allows power users to script testing batches of assembly files and logging results.

Logging is disabled by default in the GUI mode, whereas the headless mode has been designed specifically for logging.

Logging Modes

Three logging modes are supported, as described below.

All

In this mode, the emulator state is logged after each line of code executed. This does not include lines that are purely comments or pre-processor directives such as `EQU` and `DCD`.

Completion

In this mode, the emulator state is logged only once: when the end of the file is reached, or a `STOP` opcode is encountered.

Breakpoints Only

In this mode, the emulator state is logged only when a breakpoint is reached. You can place breakpoints on the GUI by clicking in the line number area. The program will pause emulation until 'Step Forwards' or 'Execute All' is clicked.

Log Settings

Log file location

Specifies the directory where the log file will be saved. The log file will be saved in this directory as `visual_log.xml`.

Items to log

Register values

Logs all register values each time the emulator state is logged. The register name `R0-R15` and value is saved as a hexadecimal number.

Status bits

Logs the values of the status bits as a 4-bit binary number of the form `0bNZCV`.

Syntax errors

Logs the line number and error message corresponding to all syntax errors. If a syntax error is encountered, emulation is cancelled so the log file will only contain syntax errors in this case.

Runtime errors

Logs the line number and error message corresponding to a runtime fault. The runtime error is appended to the log file at the point when it occurred.

Changed memory locations

Logs words in memory that have been changed by the current instruction. Therefore it logs memory locations when a `STR{B}` or `STM` instruction is encountered and the emulator state is saved. Both the memory address and the value are saved as hexadecimal numbers.

Custom memory locations

Logs a user-specified list of words in memory each time the emulator state is saved. Unlike the changed memory locations, this option will unconditionally log memory whenever the state is saved.

Log File Structure

The XML log files are structured as follows:

```
root
--- line 1
----- code
----- register R0 value
----- register R1 value
----- ...
----- register R15 value
----- status bits value
----- memory location value
----- instruction cycle count
--- line 2
----- code
----- register R0 value
...
--- line 73
--- runtime error
```

As an example, the log file for the following program:

```
MOV R0, #5
SUB R0, R0, #4
```

in the "All" mode while logging everything would be:

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<Visual xmlns="http://bitbucket.org/salmanarif/visual">
  <line id="1">
    <code>MOV R0, #5</code>
    <register name="R0">0x5</register>
    <register name="R1">0x0</register>
    <register name="R2">0x0</register>
    <register name="R3">0x0</register>
    <register name="R4">0x0</register>
    <register name="R5">0x0</register>
    <register name="R6">0x0</register>
    <register name="R7">0x0</register>
    <register name="R8">0x0</register>
    <register name="R9">0x0</register>
    <register name="R10">0x0</register>
    <register name="R11">0x0</register>
    <register name="R12">0x0</register>
    <register name="R13">0x0</register>
    <register name="R14">0x0</register>
    <register name="R15">0x4</register>
    <NZCV>0b0000</NZCV>
    <cycles>1</cycles>
  </line>
  <line id="2">
    <code>SUB R0, R0, #4</code>
    <register name="R0">0x1</register>
    <register name="R1">0x0</register>
    <register name="R2">0x0</register>
    <register name="R3">0x0</register>
    <register name="R4">0x0</register>
    <register name="R5">0x0</register>
    <register name="R6">0x0</register>
    <register name="R7">0x0</register>
    <register name="R8">0x0</register>
    <register name="R9">0x0</register>
    <register name="R10">0x0</register>
    <register name="R11">0x0</register>
    <register name="R12">0x0</register>
    <register name="R13">0x0</register>
    <register name="R14">0x0</register>
    <register name="R15">0x8</register>
    <NZCV>0b0000</NZCV>
    <cycles>1</cycles>
  </line>
</Visual>

```

Logging via the Command Line

Via the command line, i.e. in "headless" mode, logging can be performed using the following syntax:

```

java -jar visual_headless.jar --headless inputfile [outputfile] [--t:<threshold_value>] [--td:(abort|ignore)] [--regs]
[--statusbits] [--syntax] [--runtime] [--changed] [--cycles] [--cyclemodel:<cycle_model>][--custom:<locations>]
[--breakpoints:<linenumbers>] [--mode:(all|completion|breakpoint)] [--meminitval<memory_initialisation_value>]
[--autoinstmmsize] [--meminstsize:<instruction_memory_size>] [--sp:<stack_pointer_intialisation_value>] [--memmode:
(open|strict)]

```

An example command is:

```

java -jar visual_headless.jar --headless str_test.S -t:1000 -td:abort --mode:completion --regs --changed

```

The JAR file `visual_headless.jar` is located in `Visual.app/Contents/MacOS` on Mac OS X, and in the `content` folder in the installation directory for Windows and Linux.

The above parameters are defined as follows:

- Parameters within square braces `[]` are optional.
- `--headless` - Launches the application headless mode. Without this, the application JAR will start in GUI mode. **This must be placed before any other parameters.**
- `inputfile` - The path to the input assembly file. **This must be placed after `--headless` but before any other**

parameters.

- **outfile** - The optional path to the output log file. If this is not specified, the output file will be named as follows: **inputfile_log.xml**.
- **-t:<threshold_value>** - Specifies the loop warning threshold to use to detect potential infinite loops. An example of usage is **-t:2500** which sets the threshold to **2500**. The default value is **1000**.
- **-td:(abort|ignore)** - Specifies the default action to take when a possible infinite loop warning is encountered.
 - **abort** will immediately stop execution, whereas **ignore** will continue execution.
 - Using **ignore** is dangerous as it effectively disables the infinite loop detection.
 - An example of usage is **-td:abort**.
 - If this parameter is not set, the user can select options during emulation manually.
- **--regs** - If specified, register values will be logged. Otherwise, they won't be.
- **--statusbits** - If specified, status bits will be logged. Otherwise, they won't be.
- **--changed** - If specified, memory values that were changed by an instruction will be logged with that instruction. Otherwise, they won't be.
- **--syntax** - If specified, syntax errors will be logged. Otherwise, they won't be.
- **--runtime** - If specified, runtime errors will be logged. Otherwise, they won't be.
- **--cycles** - If specified, the instruction cycle count for each instruction will be logged. Otherwise, it won't be.
- **--cyclemodel:<cycle_model>** - If specified, the cycle model is set to the file at the location specified by **cycle_model**. See the [cycle models](#) page of the user guide for details
- **--custom:<locations>** - If specified, the given memory locations will be logged each time the state is saved. **locations** should be a comma-separated list (without whitespaces) of word-aligned hexadecimal memory addresses. Examples of usage are:
 - **--custom:0x8FC0**
 - **--custom:0x8FC0,0x8FC4,0x8FC8,0x8FCC,0x8FD0**
- **--breakpoints:<linenumbers>** - When used in conjunction with the "Breakpoints only" logging mode, the state will be logged every time a line with a breakpoint is executed. Otherwise, this will have no effect on execution. **linenumbers** must be a list of comma-separated line numbers. An example of usage is **--breakpoints:17,84,90**
- **--mode:(all|completion|breakpoint)** - Specifies the logging mode to use. The modes are as defined earlier. An example of usage is **--mode:completion**. If unspecified, the default mode is **all**.
- **--meminitval:<memory_initialisation_value>** - Specifies the memory initialisation value to use as a hexadecimal number. An example of usage is **--meminitvalue:0xFFFFFFFF**. If unspecified, the default value is **0x0**.
- **--autoinstmemsize** - If specified, the instruction memory region is automatically resized in blocks of 256 bytes such as to fit the code in the minimum amount of space.
- **--meminstsize:<instruction_memory_size>** - Specifies the [instruction memory size](#) as a hexadecimal number. An example of usage is **--meminstsize:0x8000**. If unspecified, the default value is **0x10000**.
- **--sp:<stack_pointer_initialisation_value>** - Specifies the value of the stack pointer to set at the start of emulation as a hexadecimal number. An example of usage is **--sp:0xBFFFC000**. If unspecified, the default value is **0xFF000000**.
- **--memmode:(open|strict)** - Specifies the memory access mode to use. In **open** mode, any location that is not in the instruction memory region has read and write access. In **strict** mode, such locations only have read access.